МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«Тульский государственный университет»

Институт прикладной математики и компьютерных наук

ЛАБОРАТОРНАЯ РАБОТА №3

по дисциплине

Мобильное программирование

по теме

«Классы»

Выполнил: ст. гр. 220032-11 Лаукерт К.М.

Принял: к.т.н., доцент ИПМКН Лихачев И.В.

Тула 2026

Лабораторная работа №3
по теме «Классы»

Цель работы

Освоить создание классов, свойств, конструкторов, наследования и переопределения методов в языке Kotlin.

Задание

Разработать систему классов: родительский и несколько дочерних. Классы должны содержать поля, свойства, функции и несколько конструкторов. Дочерние классы должны переопределять функции базового класса. Продемонстрировать использование классов в программе.

Описание решения

Разработана система классов в предметной области электроники. Базовый класс ElectronicDevice хранит модель, производителя, уровень заряда и состояние питания. Свойство batteryLevel ограничивает значение диапазоном от 0 до 100, а isPoweredOn доступно на чтение извне и изменяется только методами powerOn и powerOff. Класс имеет основной и вторичный конструкторы, а также открытые методы description и operate.

Классы Smartphone и Laptop наследуют ElectronicDevice. Смартфон хранит признак поддержки 5G, ноутбук — размер экрана. В обоих дочерних классах определены вторичные конструкторы и переопределены методы description и operate. В main объекты хранятся в списке типа ElectronicDevice, поэтому переопределённые методы вызываются полиморфно.

Текст программы (Main.kt)

```
open class ElectronicDevice(
    val model: String,
    val manufacturer: String
) {
    var batteryLevel: Int = 0
        set(value) {
            field = value.coerceIn(0, 100)
        }
```

```
    var isPoweredOn: Boolean = false
        private set

    constructor(model: String) : this(model, "Неизвестный производитель")

    fun powerOn() {
        isPoweredOn = true
        println("$model включено")
    }

    fun powerOff() {
        isPoweredOn = false
        println("$model выключено")
    }

    open fun description(): String {
        return "Устройство: $manufacturer $model, заряд: $batteryLevel%"
    }

    open fun operate() {
        println("$model выполняет базовую работу")
    }
}

class Smartphone(
    model: String,
    manufacturer: String,
    var supports5G: Boolean
) : ElectronicDevice(model, manufacturer) {
    constructor(model: String, manufacturer: String) : this(model, manufacturer, false)

    override fun description(): String {
```

```
        val network = if (supports5G) "поддерживает 5G" else "не
поддерживает 5G"
        return "Смартфон: $manufacturer $model, $network, заряд:
$batteryLevel%"
    }

    override fun operate() {
        println("$model совершает звонок и запускает
приложения")
    }
}

class Laptop(
    model: String,
    manufacturer: String,
    var screenSize: Double
) : ElectronicDevice(model, manufacturer) {
    constructor(model: String, manufacturer: String) :
this(model, manufacturer, 15.6)

    override fun description(): String {
        return "Ноутбук: $manufacturer $model, экран:
$screenSize дюйма, заряд: $batteryLevel%"
    }

    override fun operate() {
        println("$model запускает программу для разработки")
    }
}

fun main() {
    val smartphone = Smartphone("Galaxy S24", "Samsung", true)
    smartphone.batteryLevel = 95

    val laptop = Laptop("ThinkPad E14", "Lenovo")
    laptop.batteryLevel = 120
```

```
    val devices: List<ElectronicDevice> = listOf(smartphone,
laptop)


    devices.forEach { device ->
        println(device.description())
        device.powerOn()
        device.operate()
        println("Состояние питания: ${if (device.isPoweredOn)
"включено" else "выключено"}")
        device.powerOff()
        println()
    }
}
```

Результат выполнения

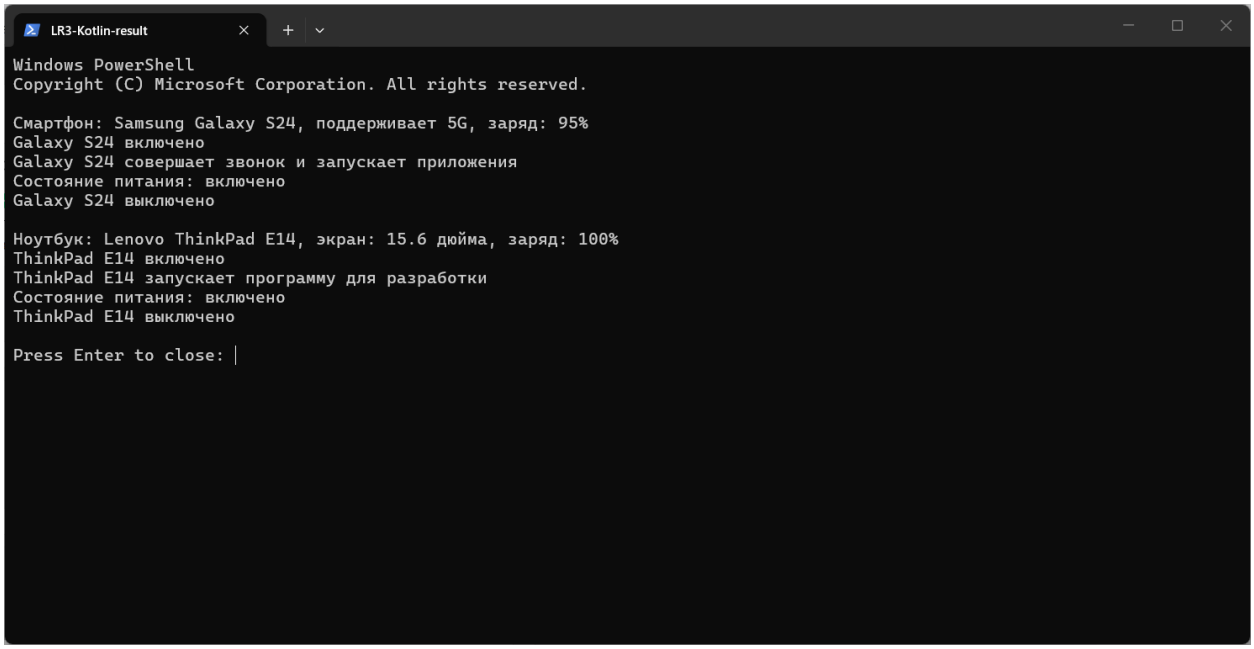


Рисунок 1 — результат выполнения программы Main.kt

Вывод

Разработана система классов для электронных устройств. В работе использованы свойства с ограничением значений, модификатор private set, основной и вторичные конструкторы, наследование и переопределение методов. Полиморфизм продемонстрирован вызовом методов дочерних объектов через ссылки на базовый класс.